

Composições e Layouts

MOV786205 – CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



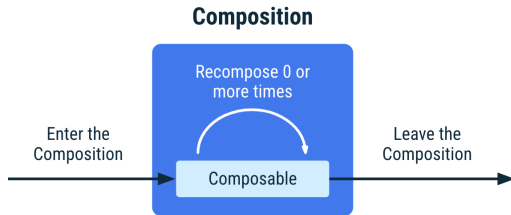
Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Assuntos da aula

- 1 Composições com Jetpack Compose e seu ciclo de vida
- 2 Componentes de interface
- 3 Layouts
- 4 Adicionar imagens no projeto e usá-las na composição Image

Composição do Jetpack Compose

- Permite criar interfaces de usuário de **forma declarativa** e reutilizável
 - Compostas por funções que retornam elementos de UI, como Text
 - São aninhadas, permitindo criar hierarquias complexas de UI
- Recomposições são acionadas quando o estado da UI muda, permitindo atualizações eficientes da interface



Fonte: (Google, 2025)

Funções de Composição

- São funções, com a anotação `@Composable`, que definem a UI
- Podem ser chamadas dentro de outras funções de composição e reutilizadas em diferentes partes da UI
- Recebem parâmetros que representam o estado da UI
- Você pode atualizar a UI alterando o estado passado para a função de composição

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

Componentes de Interface

<https://developer.android.com/develop/ui/compose/components>

- São funções de composição que representam elementos de UI
 - Text: exibe texto na tela
 - Button: representa um botão clicável
 - TextField: representa um campo de texto editável
 - Image: representa uma imagem
 - Icon: representa um ícone
 - Card: representa um cartão com sombra e borda

Funções de nível superior para criar componentes de UI

<https://developer.android.com/reference/kotlin/androidx/compose/material3/package-summary#top-level-functions>

```
@Composable
fun ExemploText() {
    Text(
        text = "Olá, Jetpack Compose!",
        fontSize = 20.sp,
        fontWeight = FontWeight.Bold,
        color = Color.Blue
    )
}
```

Olá, Jetpack Compose!

Funções de nível superior para criar componentes de UI

<https://developer.android.com/reference/kotlin/androidx/compose/material3/package-summary#top-level-functions>

```
@Composable
fun ExemploButton() {
    Button(onClick = { println("Botão clicado!") }) {
        Text("Clique aqui")
    }
}
```



Funções de nível superior para criar componentes de UI

<https://developer.android.com/reference/kotlin/androidx/compose/material3/package-summary#top-level-functions>

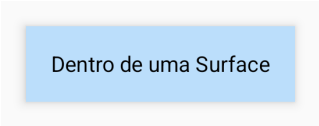
```
@Composable
fun ExemploCard() {
    Card(
        modifier = Modifier
            .padding(16.dp)
            .fillMaxWidth(),
        shape = RoundedCornerShape(8.dp),
        colors = CardDefaults.cardColors(containerColor =
            Color(0xFFE3F2FD))
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Título do Card", fontWeight = FontWeight.
                Bold)
            Text("Conteúdo do card em texto simples.")
        }
    }
}
```

Título do Card
Conteúdo do card em texto simples.

Funções de nível superior para criar componentes de UI

<https://developer.android.com/reference/kotlin/androidx/compose/material3/package-summary#top-level-functions>

```
@Composable
fun ExemploSurface() {
    Surface(
        modifier = Modifier.padding(16.dp),
        color = Color(0xFFBBDEFB),
        shadowElevation = 4.dp
    ) {
        Text("Dentro de uma Surface", modifier = Modifier.
padding(16.dp))
    }
}
```



Dentro de uma Surface

Funções de nível superior para criar componentes de UI

<https://developer.android.com/reference/kotlin/androidx/compose/material3/package-summary#top-level-functions>

```
@Composable
fun ExemploOutlinedTextField() {
    var texto by remember { mutableStateOf("") }

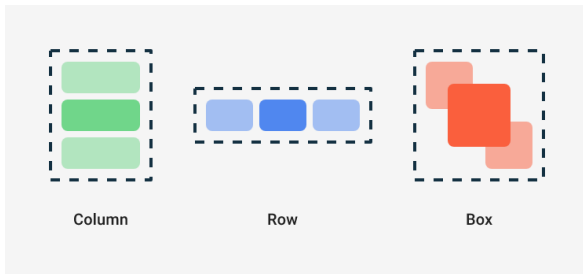
    OutlinedTextField(
        value = texto,
        onChange = { texto = it },
        label = { Text("Digite algo") },
        leadingIcon = {
            Icon(
                imageVector = Icons.Default.Search,
                contentDescription = null
            )
        },
    )
}
```

 Digite algo

Layouts

<https://developer.android.com/develop/ui/compose/layouts>

- São funções de composição que organizam outros elementos de UI
- Permitem definir como os elementos serão dispostos na tela
 - `Column`: organiza os elementos verticalmente
 - `Row`: organiza os elementos horizontalmente
 - `Box`: sobrepõe os elementos, permitindo posicionamento absoluto



Layouts

Column

```
@Composable
fun MinhasColunas() {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(text = "Olá")
        Text(text = "Mundo")
    }
}

@Preview(
    showBackground = true,
    widthDp = 320,
    backgroundColor = 0xFF89BCE5
)
@Composable
private fun MinhasColunasPreview() {
    MinhasColunas()
}
```

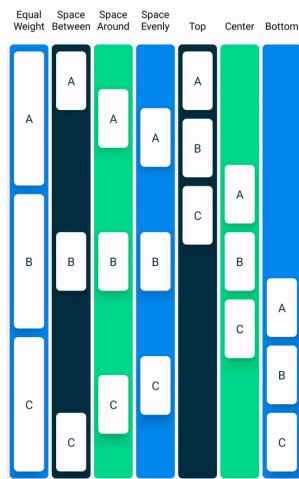


Olá
Mundo

Column: alinhamento vertical dos elementos I

O parâmetro `verticalArrangement` permite controlar o alinhamento vertical dos itens

- `Arrangement.SpaceEvenly`
- `Arrangement.SpaceBetween`
- `Arrangement.Top`
- `Arrangement.Bottom`
- `Arrangement.Center`
- `Arrangement.SpaceAround`

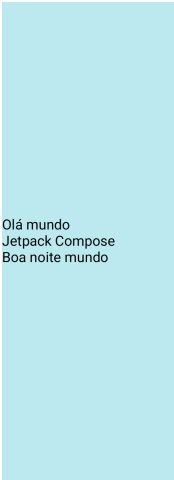


Fonte: (Google, 2025)

Column: alinhamento vertical dos elementos II

O parâmetro `verticalArrangement` permite controlar o alinhamento vertical dos itens

```
@Composable
fun ColunasArrangement(modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        // Alinhamento horizontal dos elementos
        // horizontalAlignment = Alignment.
        CenterHorizontally,
    ) {
        Text(text = "Olá mundo")
        Text(text = "Jetpack Compose")
        Text(text = "Boa noite mundo")
    }
}
```



Olá mundo
Jetpack Compose
Boa noite mundo

Column: alinhamento vertical dos elementos III

O parâmetro `verticalArrangement` permite controlar o alinhamento vertical dos itens

```
@Composable
fun ColunasArrangement(modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxSize(),
        verticalArrangement = Arrangement.
        SpaceBetween,
        // Alinhamento horizontal dos elementos
        // horizontalAlignment = Alignment.
        CenterHorizontally,
    ) {
        Text(text = "Olá mundo")
        Text(text = "Jetpack Compose")
        Text(text = "Boa noite mundo")
    }
}
```

Olá mundo

Jetpack Compose

Boa noite mundo

Modificadores de Composição

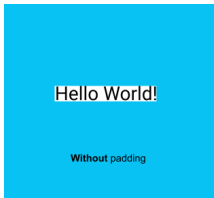
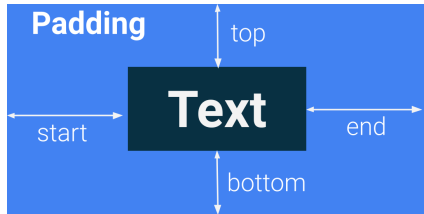
<https://developer.android.com/develop/ui/compose/modifiers>

- Modificadores permitem alterar aparência, layout, comportamento e interação dos componentes de UI no Jetpack Compose.
- São aplicados diretamente nas funções de composição, geralmente como parâmetro `modifier`.
- É uma boa prática que toda função composável aceite um parâmetro opcional `modifier`, facilitando a reutilização e personalização do componente em diferentes contextos.
- Exemplos de modificadores: `padding`, `background`, `fillMaxWidth`, `clickable`, entre outros.

Modificadores de Composição

<https://developer.android.com/develop/ui/compose/modifiers>

```
Modifier.padding(  
    start = 16.dp,  
    top = 16.dp,  
    end = 16.dp,  
    bottom = 16.dp  
)
```



Fonte: (Google, 2025)

Modificadores de Composição

<https://developer.android.com/develop/ui/compose/modifiers>

```
@Composable
fun Saudacao(nome: String, modifier: Modifier =
    Modifier) {
    Surface(
        color = MaterialTheme.colorScheme.primary,
        modifier = modifier.padding(
            vertical = 4.dp, horizontal = 8.dp)
    ) {
        Column() {
            Text(text = "Olá ")
            Text(text = nome)
        }
    }
}
```

Olá
Mundo

Olá
Compose

Modificadores de Composição

<https://developer.android.com/develop/ui/compose/modifiers>

```
@Composable
fun Saudacao(nome: String, modifier: Modifier =
    Modifier) {
    Surface(
        color = MaterialTheme.colorScheme.primary,
        modifier = modifier.padding(
            vertical = 4.dp, horizontal = 8.dp)
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(24.dp)
        ) {
            Text(text = "Olá ")
            Text(text = nome)
        }
    }
}
```

Olá
Mundo

Olá
Compose

Modificadores de Composição: peso (weight)

<https://developer.android.com/develop/ui/compose/modifiers>

```
@Composable
fun ColunasPeso(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier
        .fillMaxSize()
        .statusBarsPadding()
    ) {
        Saudacao(Modifier.weight(1f), "Mundo")
        Saudacao(Modifier.weight(2f), "Compose")
        Saudacao(Modifier.weight(1f), "Android")
    }
}

@Composable
fun Saudacao(modifier: Modifier = Modifier, nome:
    String, color: Color = MaterialTheme.colorScheme
    .primary) {
    // Conteúdo igual do slide anterior
}
```

Olá
Mundo

Olá
Compose

Olá
Android

Layout

Row

```
@Composable
fun Linha(modifier: Modifier = Modifier) {
    val mensagens = listOf("Mundo", "Compose", "Juca")
    Row() {
        for (msg in mensagens) {
            Surface(
                color = MaterialTheme.colorScheme.primary,
                modifier = modifier.padding(4.dp)
            ) {
                Text(
                    text = "Olá $msg",
                    modifier = Modifier.padding(8.dp),
                )
            }
        }
    }
}
```

Olá Mundo

Olá Compose

Olá Juca

Row: Alinhamento Horizontal

parâmetro `horizontalAlignment`

Equal Weight	
Space Between	
Space Around	
Space Evenly	
End (LTR)	
Center	
Start (LTR)	

Fonte: (Google, 2025)

Layouts

Box – Permite sobrepor elementos, posicionando-os de forma absoluta

```
fun Caixa(modifier: Modifier = Modifier) {  
    Surface() {  
        Box(  
            modifier = modifier  
                .padding(8.dp)  
        ) {  
            Text(  
                text = "Olá mundo",  
            )  
            Text(  
                text = "Jetpack Compose",  
            )  
            Text(  
                text = "Boa noite mundo",  
                fontSize = 20.sp,  
            )  
        }  
    }  
}
```

Olá mundo
Jetpack Compose
Boa noite mundo

Layouts

Box – Permite sobrepor elementos, posicionando-os de forma absoluta

```
fun Caixa(modifier: Modifier = Modifier) {  
    Surface() {  
        Box(  
            modifier = modifier.padding(8.dp)  
        ) {  
            Text(  
                text = "Olá mundo",  
                modifier = Modifier.align(Alignment.TopEnd)  
            )  
            Text(  
                text = "Jetpack Compose",  
                modifier = modifier  
                    .align(Alignment.BottomStart),  
            )  
            Text(  
                text = "Boa noite mundo",  
                fontSize = 20.sp,  
                modifier = modifier  
                    .padding(24.dp)  
                    .align(Alignment.BottomCenter)  
            )  
        }  
    }  
}
```

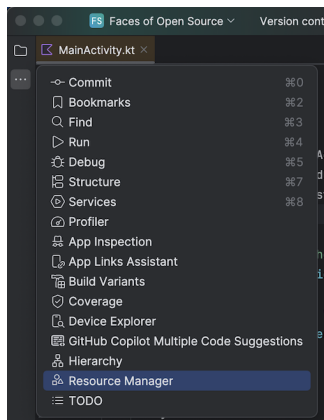
Olá mundo

Boa noite mundo

Jetpack Compose

Imagens no Projeto

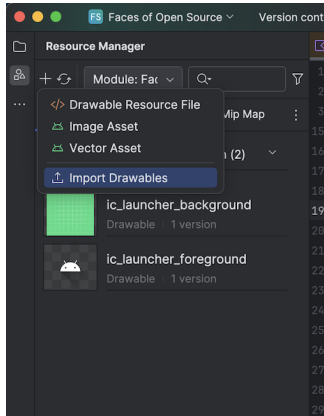
<https://developer.android.com/develop/ui/compose/graphics>



- No Android Studio, clique no ícone de reticências e selecione Resource Manager
- Clique no botão + e selecione Import Drawables
- Navegue até a pasta onde estão as imagens e selecione as que deseja adicionar
- As imagens serão copiadas para a pasta `res/drawable` do seu projeto

Imagens no Projeto

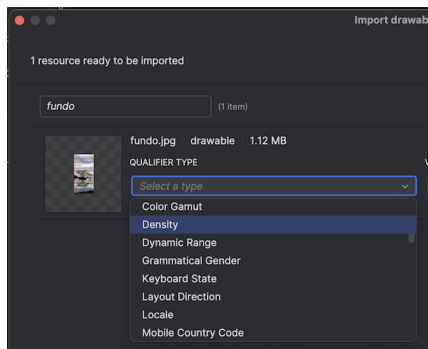
<https://developer.android.com/develop/ui/compose/graphics>



- No Android Studio, clique no ícone de reticências e selecione Resource Manager
- Clique no botão + e selecione Import Drawables
- Navegue até a pasta onde estão as imagens e selecione as que deseja adicionar
- As imagens serão copiadas para a pasta res/drawable do seu projeto

Imagens no Projeto

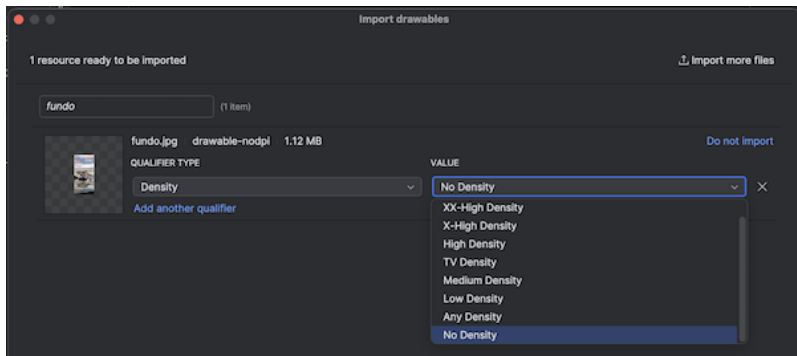
Essa será uma imagem de fundo e assim não precisa de diferentes densidades



- Em Qualifier Type escolha Density e em Value escolha No Density
- Isso evitará que seja criadas diferentes versões da imagem para diferentes densidades de tela

Imagens no Projeto

Essa será uma imagem de fundo e assim não precisa de diferentes densidades



- Em Qualifier Type escolha Density e em Value escolha No Density
- Isso evitará que seja criadas diferentes versões da imagem para diferentes densidades de tela

Imagem de fundo

<https://drive.google.com/drive/folders/1NovKCtjbyjaf8z3R93dmcgvjVKWH14GW>

- Recursos são acessados através de identificadores gerados automaticamente na classe R
 - Ex: `R.drawable.nome_da_imagem`

```
@Composable
fun Fundo(modifier: Modifier = Modifier) {
    Surface(modifier = modifier.fillMaxSize()) {
        Image(
            painter = painterResource(id = R.drawable.fundo),
            contentDescription = null,
        )
    }
}

@Preview(showBackground = true)
@Composable
private fun FundoPreview() {
    Fundo()
}
```



Imagem de fundo

- Recursos são acessados através de identificadores gerados automaticamente na classe R
 - Ex: `R.drawable.nome_da_imagem`

```
@Composable
fun Fundo(modifier: Modifier = Modifier) {
    Surface(modifier = modifier.fillMaxSize()) {
        Image(
            painter = painterResource(id = R.drawable.fundo),
            contentDescription = null,
            contentScale = ContentScale.Crop, // preencher o espaço
            alpha = 0.6F, // opacidade da imagem
        )
    }
}

@Preview(showBackground = true)
@Composable
private fun FundoPreview() {
    Fundo()
}
```



Aplicação final

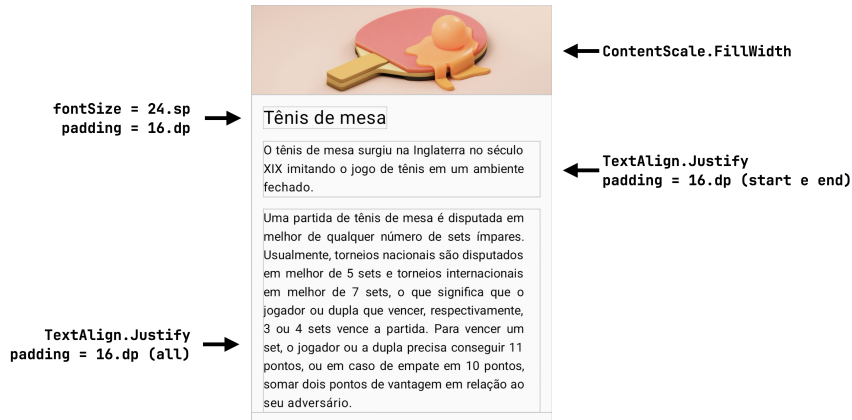
- Image e Saudacao devem ser chamadas dentro de um Box

```
@Composable
fun Saudacao(message: String, modifier: Modifier = Modifier) {
    Column(modifier = Modifier,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = message,
            fontSize = 100.sp,
            lineHeight = 116.sp,
            textAlign = TextAlign.Center
        )
        Text(
            text = "ADS",
            fontSize = 36.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier
                .padding(16.dp)
                .align(alignment = Alignment.End)
        )
    }
}
```



Desafio 01

Use o `verticalArrangement = Arrangement.Top` na **Column** para posicionar o texto no topo da tela



■ https://drive.google.com/drive/folders/1bfWhvandsuEHoEi2EH9VUtl12hQdL7_

Aula baseada em



GOOGLE. **Android Developer**. 2025. Disponível em: <https://developer.android.com/>.
Acesso em: 9 jul. 2025.